

Smart Parking POC Technical Implementation Plan

Contents

Smart Parking Availability & Prediction System	1
Proof-of-Concept — Hyper-Detailed Technical Implementation Plan	1
0. POC Framing & Ground Rules	1
1. System Architecture (Technical View)	2
2. Repository Structure	2
3. Data Model (Postgres Schema)	3
4. Sensing Layer — Technical Detail	5
4.1 Occupancy Detection	5
4.2 ANPR / Progressive Matching	5
5. Integration Layer — Status State Machine	5
6. Synthetic Data Generator (critical for a credible POC)	6
7. Prediction Layer	6
7.1 Feature Set (first version)	6
7.2 Baseline Model (Phase A — always available)	6
7.3 Trained Model (Phase B)	7
7.4 Model Serving	7
8. Backend API (FastAPI)	7
9. Frontend (React)	7
10. Local Dev / Demo Environment	8
11. Testing Strategy	8
12. Mapping to the Original Phased Roadmap	8
13. Suggested Build Order (first 2 weeks, concretely)	8

Smart Parking Availability & Prediction System

Proof-of-Concept — Hyper-Detailed Technical Implementation Plan

Version 1.0 | Draft for Internal Build Reference

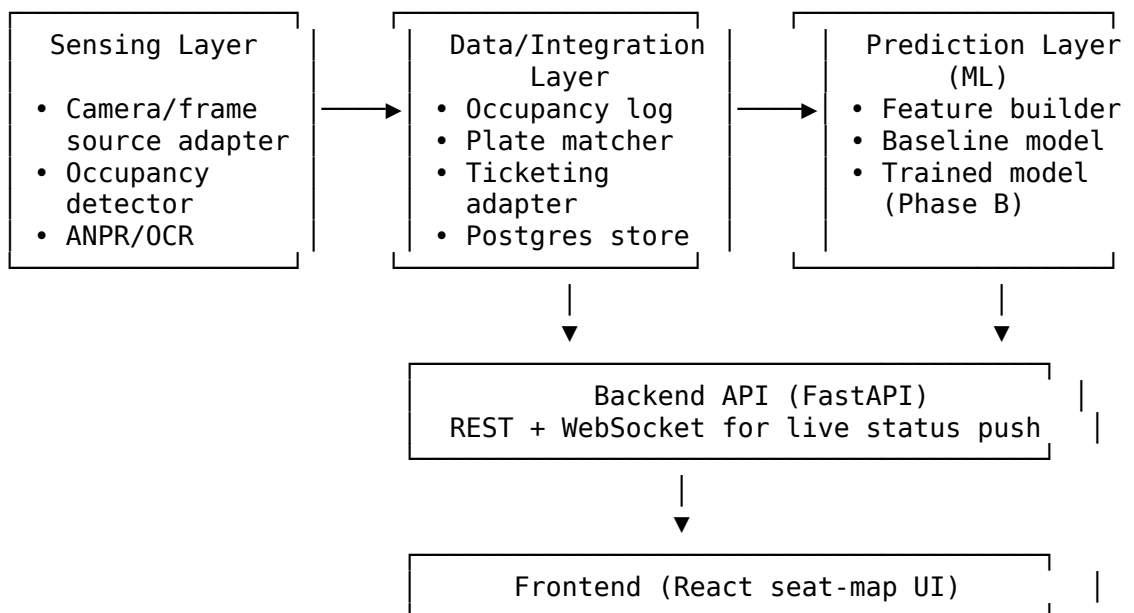
0. POC Framing & Ground Rules

This plan assumes the POC must be demonstrable **without dependency on real Megaworld infrastructure** (live cameras, live ticketing system access) being guaranteed on day one. Every physical-world input (camera feed, ticketing records) is built behind an **adapter interface**, with a synthetic/simulated implementation shipped first and a “real” implementation stubbed for later swap-in. This de-risks the timeline: if real camera/ticketing access never materializes

during the POC window, the demo still runs end-to-end on simulated data with the exact same code path that would run on real data.

POC Success Criteria (concrete, testable): 1. A seat-map UI renders live slot status (Free / Occupied-Unpaid / Likely Vacating Soon) updating within ~5 seconds of a simulated (or real, if available) camera frame change. 2. A plate-to-payment matching demo correctly resolves at least 90% of simulated “clean” reads and correctly falls back to “unresolved” on ambiguous/low-confidence reads (never silently guesses wrong). 3. A trip-planning endpoint returns a probability/estimate of availability for a future timestamp, backed by either (a) a trained model on ≥ 2 -4 weeks of collected/simulated historical data, or (b) an explicitly-labeled heuristic baseline if data volume is insufficient. 4. The whole pipeline (camera/simulator $\rightarrow$ occupancy $\rightarrow$ matching $\rightarrow$ prediction $\rightarrow$ UI) runs as one demoable system, not disconnected scripts.

1. System Architecture (Technical View)



Key architectural decision: adapter pattern at every external boundary. - Camera-Source interface $\rightarrow$ SimulatedCameraSource (POC) / RTSPCameraSource (future) - Ticketing-Source interface $\rightarrow$ SimulatedTicketingSource (POC) / MegaworldTicketingAdapter (future)

This means the occupancy detector, matcher, and predictor never know or care whether their input is synthetic or real — swapping the data source is a one-line config change, not a code change.

2. Repository Structure

```
smart-parking-poc/
├── sensing/
│   ├── camera_source.py          # CameraSource ABC + SimulatedCameraSource
│   ├── occupancy_detector.py     # slot-region occupancy inference
│   └── anpr.py                   # plate OCR wrapper (candidate-narrowing logic)
```

```

├── calibration/
│   └── slot_regions.json      # per-camera slot bounding-box config
├── integration/
│   ├── ticketing_source.py    # TicketingSource ABC + SimulatedTicketingSource
│   ├── matcher.py            # progressive plate-to-payment matching
│   └── status_engine.py       # slot status state machine (Section 7.1 of original doc)
├── prediction/
│   ├── features.py           # feature engineering (time, calendar, events)
│   ├── baseline_model.py     # heuristic hourly/daily average model
│   ├── train.py              # gradient-boosted model training script
│   └── predict.py            # inference wrapper (baseline/trained switch)
├── data/
│   ├── synthetic_generator.py # generates plausible occupancy + ticketing streams
│   ├── holiday_calendar.json
│   └── events_calendar.json
├── api/
│   ├── main.py               # FastAPI app
│   ├── routers/
│   │   ├── slots.py
│   │   ├── predictions.py
│   │   └── ws_live.py
│   └── models.py             # Pydantic schemas
├── db/
│   ├── schema.sql
│   └── migrations/
├── frontend/
│   └── (React app – see Section 8)
├── tests/
├── docker-compose.yml
└── README.md

```

3. Data Model (Postgres Schema)

```

-- Static configuration
CREATE TABLE sites (
    site_id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,           -- e.g. 'Eastwood City'
    timezone TEXT NOT NULL DEFAULT 'Asia/Manila'
);

CREATE TABLE zones (
    zone_id SERIAL PRIMARY KEY,
    site_id INT REFERENCES sites(site_id),
    level TEXT NOT NULL,          -- 'Basement 2'
    label TEXT NOT NULL,          -- 'Zone A'
    total_capacity INT NOT NULL
);

CREATE TABLE slots (
    slot_id SERIAL PRIMARY KEY,
    zone_id INT REFERENCES zones(zone_id),

```

```

    slot_code TEXT NOT NULL,           -- 'B2-A-014'
    camera_id TEXT NOT NULL,
    region_coords JSONB NOT NULL      -- calibration bbox for this slot in camera frame
);

-- Live + historical state
CREATE TABLE occupancy_events (
    event_id BIGSERIAL PRIMARY KEY,
    slot_id INT REFERENCES slots(slot_id),
    status TEXT NOT NULL CHECK (status IN ('free','occupied')),
    detected_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    confidence REAL                    -- occupancy model confidence
);

CREATE TABLE plate_reads (
    read_id BIGSERIAL PRIMARY KEY,
    slot_id INT REFERENCES slots(slot_id),
    raw_ocr_text TEXT,                -- ephemeral, purge policy applies (Section 10)
    char_confidences JSONB,           -- per-character confidence array
    captured_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ticketing_records (
    ticket_id TEXT PRIMARY KEY,       -- from ticketing system (or simulator)
    plate_partial TEXT,               -- last known partial/full plate on file
    entry_time TIMESTAMPTZ,
    payment_settled_at TIMESTAMPTZ,
    site_id INT REFERENCES sites(site_id)
);

CREATE TABLE plate_matches (
    match_id BIGSERIAL PRIMARY KEY,
    slot_id INT REFERENCES slots(slot_id),
    ticket_id TEXT REFERENCES ticketing_records(ticket_id),
    match_confidence REAL NOT NULL,
    resolved BOOLEAN NOT NULL,       -- false = below threshold, left unresolved
    matched_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE slot_status_current (
    slot_id INT PRIMARY KEY REFERENCES slots(slot_id),
    status TEXT NOT NULL CHECK (status IN
        ('free','occupied_unpaid','occupied_likely_vacating')),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Prediction inputs
CREATE TABLE holidays (date DATE PRIMARY KEY, name TEXT);
CREATE TABLE events (
    event_id SERIAL PRIMARY KEY,
    site_id INT REFERENCES sites(site_id),
    name TEXT, starts_at TIMESTAMPTZ, ends_at TIMESTAMPTZ,
    expected_impact TEXT             -- 'high'/'medium'/'low', manually tagged for POC
);

```

```
);
```

Retention note (ties to privacy section of the original doc): `plate_reads.raw_ocr_text` should be purged on a rolling job (e.g., nightly delete of rows older than 24-48h) once a `plate_matches` row has been resolved from it — the match record persists, the raw text does not. For the POC this can just be a cron/script; for production this would need a proper retention policy.

4. Sensing Layer — Technical Detail

4.1 Occupancy Detection

- **POC approach:** rather than standing up a full CV pipeline against real cameras (which you don't have guaranteed access to), build `SimulatedCameraSource` that replays a labeled video clip or synthetic frame sequence with known ground-truth occupancy, so the detector's output can be scored against a real answer key.
- **If real footage becomes available:** use a lightweight pretrained object detector (e.g., a small YOLO-family model) fine-tuned or zero-shot on "vehicle present in region" per calibrated bounding box — this is a presence/absence classification per region, not full-frame detection, which keeps compute low.
- **Calibration tool:** a simple script/notebook where a human clicks 4 corner points per slot on a reference frame → stored as `region_coords` in `slots.region_coords`. One-time per camera install.
- **Output cadence:** poll every 5-15 seconds for the POC (real-time-enough for a demo, cheap on compute).

4.2 ANPR / Progressive Matching

- OCR component: any off-the-shelf plate OCR library (e.g., EasyOCR or a lightweight ANPR model) run only on the cropped slot region, not the full frame — keeps it fast and reduces false reads from unrelated plates.
 - **Matching algorithm (this is the interesting part — implement carefully):**
 1. Pull the current candidate pool = `ticketing_records` for this site where `payment_settled_at` IS NOT NULL and not yet matched to a free transition.
 2. For each OCR read, compute a weighted edit-distance against each candidate's `plate_partial`, where substitution cost is scaled by $(1 - \text{char_confidence})$ and known confusable pairs (O/O, I/l, 8/B, 5/S, 2/Z) get a reduced substitution cost.
 3. Rank candidates by score. Accept if top score clears an absolute threshold **and** has a minimum margin over the second-best candidate (avoids near-tie false accepts).
 4. Otherwise, write a `plate_matches` row with `resolved = false` — slot stays `occupied_unpaid` in the state machine.
 - This whole algorithm is unit-testable in isolation with synthetic OCR strings — no camera needed to validate correctness, which is valuable for a POC demo.
-

5. Integration Layer — Status State Machine

Implement `status_engine.py` as an explicit finite-state machine (not scattered if/else logic), driven by two event streams (occupancy transitions, match events):

FREE —(occupancy: present)→ OCCUPIED_UNPAID

OCCUPIED_UNPAID —(match resolved, above threshold)→ OCCUPIED_LIKELY_VACATING

OCCUPIED_LIKELY_VACATING —(occupancy: absent)→ FREE
OCCUPIED_UNPAID —(occupancy: absent)→ FREE [vehicle left without a resolved match – fine, still valid]

Every transition writes to `slot_status_current` and also appends to `occupancy_events` / `plate_matches` for the historical log the prediction layer consumes later.

6. Synthetic Data Generator (critical for a credible POC)

Since real historical data won't exist on day one, build `synthetic_generator.py` to produce a plausible multi-week dataset:

- Base occupancy curve per zone: parameterized daily curve (e.g., low overnight, ramp at 10am, lunch peak 12-1pm, after-office peak 6-8pm, weekend midday peak) plus Gaussian noise.
- Inject holiday/event multipliers on specific dates (pull from holidays / events tables) — e.g., +40% occupancy on a tagged “concert” event day.
- Simulate matching noise: some fraction of plate reads are deliberately corrupted at the character level to exercise the confidence-weighted matcher realistically.
- **Label all generated data as synthetic** (a `is_synthetic` BOOLEAN flag or separate schema) so the demo can honestly show “this model is trained on simulated data” rather than implying real historical accuracy — this maps directly to the Risks section of the original plan (Insufficient historical data for ML model).

This generator is arguably the single most important POC artifact: it's what makes Phases 3 and 4 demoable even with zero real infrastructure access.

7. Prediction Layer

7.1 Feature Set (first version)

Feature	Type	Source
hour_of_day, day_of_week	categorical/cyclical	timestamp
is_holiday	boolean	holidays table
is_weekend	boolean	derived
event_active, event_impact_tier	boolean/categorical	events table
rolling_avg_occupancy_same_hour_last_4_weeks	numeric	occupancy_events aggregation
zone_capacity	numeric	zones.total_capacity

7.2 Baseline Model (Phase A — always available)

Simple grouped historical average: mean occupancy rate for (`zone_id`, `day_of_week`, `hour_of_day`), computed from whatever data exists (synthetic or real). Trivial to implement (`pandas.groupby`), always has an answer even with 1 day of data, and is the fallback `predict.py` uses when the trained model isn't ready or confidence is low.

7.3 Trained Model (Phase B)

- Gradient-boosted trees (e.g., LightGBM or XGBoost) regressing on occupancy rate at a given zone/timestamp, using the feature set above.
- Rationale: tabular, seasonal, relatively low-volume data — boosted trees outperform deep learning here and are far easier to explain in a POC presentation.
- Train/validation split: time-based (train on weeks 1-3, validate on week 4), not random shuffling — random splits would leak future patterns into training and overstate accuracy.
- Conservatism requirement from the original plan (never over-promise “available”): calibrate the model’s output probability, and apply a small downward adjustment bias so borderline predictions lean toward “occupied” rather than “free” — false optimism is worse than false pessimism per the doc.

7.4 Model Serving

`predict.py` exposes one function: `predict_availability(zone_id, target_timestamp) -> {probability, source: "baseline"|"trained", confidence}`. The API layer calls this directly (no need for a separate model-serving service at POC scale).

8. Backend API (FastAPI)

Endpoint	Method	Purpose
<code>/sites/{site_id}/zones</code>	GET	list zones + current capacity
<code>/zones/{zone_id}/slots</code>	GET	current status of every slot in a zone
<code>/zones/{zone_id}/slots/ws</code>	WebSocket	push live status changes for the seat-map UI
<code>/predict</code>	GET	?zone_id=&target_time= → availability estimate
<code>/admin/simulate/tick</code>	POST	(POC-only) manually advance the simulated camera/ticketing clock for demo control

Use Pydantic models for all request/response schemas; keep the WebSocket payload minimal (slot_id, new status only — client already has full slot metadata cached).

9. Frontend (React)

- **Seat-map component:** grid layout per level, one cell per slot, color-coded by status (green=free, red=occupied-unpaid, amber=likely-vacating). Subscribes to the `/slots/ws` channel and patches individual cells on message — no full re-fetch per update.
- **Trip-planning view:** a simple time picker + zone selector calling `/predict`, rendering the probability as a plain-language estimate (“Likely available” / “Uncertain” / “Unlikely”) rather than a raw percentage, matching the original doc’s UX intent.
- Libraries: plain React + a small CSS grid is sufficient for a POC — no need for a heavy charting library except a simple line/bar chart on an internal “model performance” debug view if you want one for the presentation.

10. Local Dev / Demo Environment

`docker-compose.yml` services: 1. `postgres` — schema from Section 3 2. `api` — FastAPI app 3. `simulator` — background worker running `synthetic_generator.py` on a loop, writing occupancy/ticketing events at an accelerated clock rate (e.g., 1 simulated hour per 10 real seconds) so a multi-week dataset can be generated in minutes for the demo, and live behavior is still watchable in real time during a live demo run. 4. `frontend` — React dev server

This means the entire POC can be spun up with `docker-compose up` on a laptop with no external dependencies — important if Megaworld infra access is delayed or unavailable during the internship window.

11. Testing Strategy

- **Unit tests:** matcher scoring logic (Section 4.2) with hand-crafted OCR-confusion cases; state machine transitions (Section 5); baseline model math.
- **Integration test:** run the full simulated pipeline for a short synthetic window and assert the final `slot_status_current` table matches expected ground truth from the generator.
- **Model validation:** time-based holdout accuracy + a manual sanity check that predictions never claim “available” during known synthetic peak-hour windows (calibration check tied to Section 7.3’s conservatism requirement).

12. Mapping to the Original Phased Roadmap

Original Phase	Technical Deliverable Here
Phase 0 (Discovery)	Repo scaffold, DB schema, adapter interfaces defined
Phase 1 (Live Occupancy MVP)	<code>SimulatedCameraSource</code> + occupancy detector + seat-map UI, live via <code>WebSocket</code>
Phase 2 (Ticketing Cross-Reference)	<code>SimulatedTicketingSource</code> + matcher + status state machine
Phase 3 (Historical Data Collection)	<code>synthetic_generator.py</code> producing multi-week logged dataset
Phase 4 (Prediction v1)	Baseline model always on; trained <code>LightGBM</code> model once synthetic data window is sufficient
Phase 5 (Integration & Demo)	<code>/predict</code> endpoint + trip-planning UI view + <code>docker-compose</code> one-command demo

13. Suggested Build Order (first 2 weeks, concretely)

1. DB schema + `docker-compose` skeleton (day 1-2)
2. `CameraSource/TicketingSource` adapter interfaces + simulated implementations (day 2-4)
3. Status state machine + matcher logic with unit tests (day 4-6)
4. FastAPI endpoints + `WebSocket` push, wired to the state machine (day 6-8)
5. Seat-map React UI consuming live `WebSocket` data (day 8-10)

6. Synthetic historical generator running in background, accumulating data (day 10-12, ongoing)
7. Baseline prediction model + /predict endpoint (day 12-14)

Trained model (LightGBM) and trip-planning UI polish follow once the baseline loop is provably working end-to-end.